

AIKUKISNA

Seguridad y Buenas Prácticas

- Protección de rutas/datos (roles y permisos)
 - Manejo de errores
- Manejo de estados (expiración de sesión)
 - Autenticación de 2 factores
 - Validación de entradas
 - Desarrollo seguro

Hackathon Nicaragua 2026 (HN10)

Equipo YAWANSATECH

Seguridad y Buenas Prácticas

Este documento reúne las prácticas de seguridad que implementamos en Aikukisna a lo largo del desarrollo, verificadas directamente sobre el proyecto en producción y no solo a nivel de diseño. Cada sección incluye, cuando aplica, un ejemplo real, código, resultado de una prueba, o configuración confirmada que respalda lo descrito.

Práctica	Estado
Row Level Security (RLS)	Activo en las 16 tablas de la base de datos
Contenido de solo lectura	Diccionario, lecciones y cultura: lectura pública, sin permisos de escritura para anon/authenticated
Datos de usuario protegidos	anon sin ningún acceso; authenticated solo puede leer y modificar su propia fila
Tabla de superusuario aislada	super_administrador sin acceso desde la API pública
Permisos por defecto controlados	ALTER DEFAULT PRIVILEGES configurado para que tablas nuevas no hereden permisos amplios por accidente
Manejo de errores	Todas las operaciones de red en la capa data están envueltas en try/catch, devolviendo el mensaje real del error sin exponer detalles internos innecesarios

1. Protección de rutas y datos (roles y permisos) y desarrollo seguro

Row Level Security (RLS) activo en las 16 tablas de producción, sin excepciones, con 19 políticas verificadas. La escritura de campos sensibles no pasa por el cliente:

- XP, racha actual y racha máxima están protegidos por un trigger de base de datos que revierte cualquier intento de escritura directa.
- El desbloqueo de logros se valida en el servidor mediante una función SECURITY DEFINER que verifica la condición real (lecciones completadas, racha, etc.) antes de otorgarlo — el cliente no puede auto-asignarse logros.
- Permisos mínimos por rol (anon vs. authenticated): el contenido educativo es de lectura pública; los datos personales exigen `auth.uid() = id`.

Ejemplo:

```
-- Intento directo de escritura, simulando un cliente autenticado:
```

```
UPDATE usuario SET xp = xp + 100 WHERE id = '<usuario>';
```

```
-- Resultado verificado: xp se mantuvo en 0.
```

```
-- El trigger revirtió el cambio antes de guardarlo.
```

2. Manejo de errores

Cada operación crítica de autenticación, datos y red está envuelta en manejo de excepciones, con estados de error explícitos por pantalla (carga, éxito, error) en lugar de fallos silenciosos. Verificado en vivo contra escenarios reales, incluyendo rechazos de permisos y errores de red.

Ejemplo:

```
try {  
    val id = iniciarSesionUseCase(correo, contrasena)  
    idSesionActual = id  
} catch (e: Exception) {  
    errorMessage = e.message ?: "Error al conectar con el servidor"  
}
```

3. Manejo de estados (expiración de sesión)

Configuración de Supabase Auth verificada directamente en el proyecto:

- Token de acceso válido por 1 hora, con renovación automática y transparente para el usuario.
- Detección activa de reuso de refresh tokens, que previene ataques de repetición si un token es robado.
- La expiración de sesión por inactividad requiere el plan Pro de Supabase (no disponible en el plan actual del proyecto); se documenta como limitación de infraestructura conocida, no como omisión de diseño.

Ejemplo:

Access token expiry time: 3600 segundos

Refresh token reuse detection: activado

Time-box / inactivity timeout: bloqueado por plan (requiere Pro)

4. Autenticación de dos factores (2FA)

Evaluada y diseñada a nivel de arquitectura: interfaz de repositorio y cinco casos de uso (enrolamiento, confirmación, verificación de necesidad de segundo factor, verificación de código, desenrolamiento) construidos sobre el soporte nativo de Supabase Auth para TOTP. No se integró a la interfaz de usuario en esta versión: el público objetivo son estudiantes, y el enrolamiento de TOTP requeriría que un padre o tutor lo gestione en nombre del usuario final, generando fricción sin un beneficio de seguridad proporcional para este perfil de usuario.

Ejemplo:

```
val (actual, siguiente) = client.auth.mfa.getAuthenticatorAssuranceLevel()
```

```
val necesitaSegundoFactor =  
    actual == AuthenticatorAssuranceLevel.AAL1 &&  
    siguiente == AuthenticatorAssuranceLevel.AAL2
```

5. Validación de entradas

Implementada en la capa de dominio (no solo en la interfaz), de forma que ningún caso de uso puede saltarse la validación sin importar qué pantalla lo invoque:

- Formato de correo electrónico verificado antes de enviarlo al servidor.
- Longitud mínima de contraseña (6 caracteres).
- Campos obligatorios (nombre, nombre de usuario) no vacíos.
- Rangos válidos para datos numéricos (edad).

Ejemplo:

```
require(correo.isNotBlank() && PATRON_CORREO.matches(correo)) {  
    "El correo no tiene un formato válido"  
}  
  
require(contrasena.length >= 6) {  
    "La contraseña debe tener al menos 6 caracteres"  
}
```

6. Desarrollo seguro

Las claves de API de Gemini (IA conversacional) y ElevenLabs (síntesis de voz) fueron migradas de la aplicación cliente a Edge Functions de Supabase, ejecutadas del lado del servidor:

- Ninguna clave de API viaja dentro del APK ni es extraíble por ingeniería inversa.
- Ambas funciones exigen un usuario autenticado real — no basta con la clave anónima pública de la aplicación — verificando el token contra Supabase Auth en cada petición.
- Comportamiento verificado en producción con evidencia de logs del servidor: peticiones sin sesión son rechazadas (401); peticiones con sesión válida son procesadas correctamente (200).
- Historial completo de control de versiones auditado: ninguna clave o secreto quedó expuesto en el repositorio en ningún commit de ninguna rama.

Ejemplo:

POST	401	.../functions/v1/gemini-proxy	(sin sesión → rechazado)
POST	200	.../functions/v1/gemini-proxy	(con sesión → procesado)
POST	401	.../functions/v1/sintetizar-voz	(sin sesión → rechazado)
POST	200	.../functions/v1/sintetizar-voz	(con sesión → procesado)